



Author:
Henrik Gerdes

Tags:

Grafana 2

Kubernetes 7

Monitoring 2

Recent Articles:

Most Recent

Rootless GitLab Runners

Read Now

22/05/2025

Follow Up: Let's talk about anonymous access to Kubernetes

Read Now

20/04/2025

Level up your Ansible Code - Creating Golden Images

Read Now

12/02/2025

Understanding and using modern day authentication frameworks to improve security, productivity and user acceptance

Read Now

10/11/2024

What is new in containerd 2.0

Read Now

28/10/2024

Making OnPrem Kubernetes feel like AKS/EKS/GKE

Read Now

01/09/2024

New Website - Abandon JavaScript

 Published: 14/11/2025

 6 minute read

I can't recommend Grafana anymore

Disclaimer: This tells my personal experiences with Grafana products. It also includes some facts but your experience may entirely vary and I would love to hear your take.

I started my work life at a small software company near my university. They develop, run websites and operate web services for multiple clients. Everyone had multiple responsibilities, and they heavily relied on interns and freshmen—which can be both bad and good.

For me it was good because I learned a lot.

At some point we needed a monitoring solution, and Zabbix didn't fit well into the new and declarative world of containers and Docker. I was tasked to find a solution. I looked at Loki/Prometheus with Grafana and Elastic with Kibana. Elastic was a beast! Heavy, hard to run, resource-hungry, and complex, Loki and Prometheus were the perfect fit back then.

So I created a `docker-compose.yaml` with Loki, Prometheus and Grafana. Since they all had the internal Docker network, we required no auth between them. Grafana was only exposed over an SSH tunnel. One static scrape config and the Docker Loki log plugin later, we had our observability stack. For non-Docker logs, we used Promtail. Loki and Prometheus stayed on the same machine and all we required was a local volume mount. Load was minimal.

 This is when I learned that you should not transform every log parameter to a label just to make it easier to select in the Grafana UI. Having a label for latency with basically limitless values will fill every disk's inodes, that's just how Cortex bin-packs.

I also found out Grafana Labs has a cloud offering with a nice free tier. So I even used this for personal stuff. I had a good experience with them.

Time goes on, and I switched jobs. Now we have Kubernetes.

The Prometheus container was now switching nodes. Remote storage was a problem back then, and also our workload increased by a lot. We also needed long-term storage (13 months). So I looked around and found Thanos and Mimir.

Previous experiences with Grafana products were good, so I chose Mimir. Should be similar to Loki since both are based on Cortex. Now we didn't really need Prometheus anymore. We were only using `remote_write` from Prometheus. Grafana had a solution for this. With the Grafana Agent, you can ship both logs and metrics to a remote location all in one binary. This seemed like a no-brainer.

Time goes on, and Grafana changed the Grafana Agent setup to Grafana Agent Flow Mode—some adjustments, but okay - software changes. And man, did Grafana like to change things.

They started to build their own observability platform to steal some of DataDog's customers. They created Grafana OnCall their own notification system. Not only that, but they heavily invested in Helm charts and general starter templates. Basically two commands to install the metric/log shippers and use Grafana Cloud. And even when you don't want or can't use Grafana Cloud, here are the Helm charts to install Mimir/Loki/Tempo. To make things even easier, let's all put it in an umbrella chart (it renders to 6k lines in the default state). Or use their Grafana Operator to manage Grafana installs - or at least parts of it.

As many may have experienced, software maintenance shows with age.

Grafana OnCall is deprecated, and Grafana Agent and Agent Flow were deprecated within 2-3 years of their creation. Some of the *easy to use* Helm charts are not maintained anymore. They also deprecated Angular within Grafana and switched to React for dashboards. This broke most existing dashboards.

On the same day they deprecated the Grafana Agent, they announced Grafana Alloy. The all in one replacement. It can do Logs, Metrics, Traces (zipkin & jaeger) and OTEL. The solution for

Read Now

01/07/2024

Benchmarking what actually drive our containers

Read Now

12/06/2024

Using GitLab to manage Kubernetes access

Read Now

18/05/2024

The recurring problem of the Kubernetes metrics server and insecure Kubelet certificate

Read Now

18/03/2024

Using AWS from GitHub without Credentials

Read Now

31/01/2024

Comparing GitHub Actions with GitLab CI/CD - A deep dive!

Read Now

12/12/2023

Building preconfigured OS images with HashiCorp Packer

Read Now

09/07/2023

Building and Cracking the Enigma - Part I

Read Now

everything!

The solution kind of had a rough start and was a *little* buggy. But it got better over time. The Alloy Operator also entered the game because why not.

i They choose to use their own configuration language for alloy. Something that looks like HCL. I can understand why thy didn't want to use YAML but I'm still not a fan of this. Not everything needs their own DSL.

Happy End, right? - Not quite

The all-in-one solution does not support everything. While Grafana built their own monitoring empire, the kube-prometheus community consistently and naturally developed. The Prometheus Operator with ServiceMonitor and PodMonitor CRDs became the defacto standards. So Alloy also supports the monitoring.coreos.com api-group CRDs, at least some parts of it. It natively works with ServiceMonitor and PodMonitor, but PrometheusRules needs extra configuration. The AlertmanagerConfig which would need to be implemented in Mimir is not supported. Because Mimir brings its own Alertmanager - at least sort of. There are version differences and small incompatibilities.

But I got it all working; now I can finally stop explaining to my boss why we need to re-structure the monitoring stack every year.

Grafana just released Mimir 3.0. They re-architected the ingestion logic for scalability, and now they use a message broker. Yes, Mimir in version 3.0 needs Apache Kafka to work.

None of the above things alone would be a reason to ditch Grafana products. Set aside the fact that they made it incredibly difficult now to find the ingestion endpoints for Grafana Cloud since they want to push users to use their new fleet-config management service. But all this together makes me uncomfortable recommending Grafana stuff.

I just don't know what will change next.

I want stability for my monitoring; I want it boring, and that's something Grafana is not offering. It seems like the pace within Grafana is way too fast for many companies, and I know for a fact that that pace is partially driven by career-driven development. There are some smart people at Grafana but not every customer is smart nor has the capacity to make Grafana their priority number one. Complexity kills - we've seen this.

i Don't get me wrong. Mimir, Loki and Grafana are technically really good software products and I (mostly) still like them but it's the way these products are managed which makes me question them.

Sometimes I wonder how I would see this if I had chosen the ELK stack at my first job. I also wonder if the OpenShift approach (kube-prometheus-stack) with Thanos for long-term storage is the most time-stable solution. I just hope OTEL settles, gets stable and boring fast, and just lets me pick whatever I want for my backend. Because right now I'm done with monitoring. I just want to support our application and do not want to revisit the monitoring setup every x weeks, because monitoring is a necessity—not the product. At least for most companies.